

React component architecture

COMPLETE LEARNING GUIDE

What You'll Learn

React's core idea is that a UI is a function of state: components declare what the UI should look like for a given set of props and state, and React handles turning that declaration into actual DOM updates. Getting comfortable with component architecture means understanding function components and JSX, how props flow data downward and composition avoids over-coupling components together, how `useState` and `useEffect` manage local state and side effects respectively, and when to extract logic into a custom hook instead of duplicating it. This guide also covers the classic 'lifting state up' pattern and common component design patterns you'll see in production codebases. These are the exact skills that separate a component tree that's pleasant to extend from one that turns into unmanageable prop-drilling spaghetti.

1. Function Components and JSX

A React function component is just a JavaScript function that returns JSX — a syntax extension that looks like HTML but compiles to `React.createElement()` calls describing the UI tree. Because it's just JavaScript, you can embed any expression inside curly braces: variables, function calls, ternaries, and array `.map()` calls for rendering lists. JSX enforces a single root element per return (or a `Fragment`, `<>...</>`, when you don't want an extra wrapper DOM node), and uses camelCase for attributes that mirror DOM properties (`className` instead of `class`, `onClick` instead of `onclick`) because they're JavaScript property names, not HTML attribute strings.

JSX

```
function StudentCard({ name, score }) {  
  const passed = score >= 60;  
  return (  
    <div className={`card ${passed ? "card--pass" : "card--fail"}`}>  
      <h3>{name}</h3>  
      <p>Score: {score}</p>  
    </div>  
  );  
}
```

2. Props and Composition

Props are read-only inputs passed from a parent to a child — a component must never reassign or mutate its own props. Composition is React's alternative to deep inheritance hierarchies: instead of a component trying to handle every possible variant internally, you compose smaller, focused components together, often passing JSX itself as a prop (most commonly as children). This is what makes a generic `<Modal>` or `<Card>` reusable across an entire app — the shell component owns layout and behavior, and the caller supplies the specific content via children, keeping the shell component ignorant of what it's actually displaying.

- `props.children` lets a wrapper component render whatever JSX its caller passes between its opening and closing tags.
- Prefer composing small components over adding more and more conditional props to one large component — the latter tends to grow unmanageable.
- Destructuring props in the function signature (`function Card({ title, children })`) is the standard, more readable convention over accessing `props.title` everywhere.

3. `useState` and Local Component State

`useState(initialValue)` returns a pair: the current state value and a setter function, and calling the setter schedules a re-render with the new value on the next render pass — it does not mutate the variable in place. React compares state by reference for objects and arrays, so updating a nested field requires creating a new object/array (typically via spread syntax) rather than mutating the existing one; mutating in place and calling the setter with the same reference will not trigger a re-render at all, which is one of the most common React bugs beginners hit.

JSX

```
function TodoForm() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState("");

  function addTodo() {
    setTodos((prev) => [...prev, { id: Date.now(), text }]); // new array, not mutated
    setText("");
  }

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={addTodo}>Add</button>
    </div>
  );
}
```

4. useEffect and Side Effects

`useEffect(fn, deps)` runs `fn` after React commits the render to the DOM, and is the designated escape hatch for anything that isn't purely rendering: fetching data, subscribing to an external event source, manually manipulating a DOM node a library needs direct access to, or setting up a timer. The dependency array controls when the effect re-runs: omitting it runs the effect after every render, an empty array runs it once on mount, and a populated array re-runs it whenever any listed value changes between renders. When an effect sets up something ongoing (a subscription, a timer, an event listener), returning a cleanup function from it is mandatory — React calls that cleanup before the next effect run and on unmount.

JSX

```
function StudentProfile({ studentId }) {
  const [student, setStudent] = useState(null);

  useEffect(() => {
    let cancelled = false;
    fetch(`/api/students/${studentId}`)
      .then((r) => r.json())
      .then((data) => { if (!cancelled) setStudent(data); });

    return () => { cancelled = true; }; // avoid setting state after unmount
  }, [studentId]); // re-fetch whenever studentId changes

  if (!student) return <p>Loading...</p>;
  return <h2>{student.name}</h2>;
}
```

5. Lifting State Up and Avoiding Prop Drilling

When two sibling components need to share or stay in sync with the same piece of state, the fix is to move ('lift') that state to their nearest common ancestor and pass it down as props to both, along with a callback for children to request changes. This keeps a single source of truth instead of two components independently tracking the same data and drifting out of sync. The trade-off is prop drilling — passing a prop down through several intermediate components that don't use it themselves, just to reach a deeply nested consumer. A little drilling (two or three levels) is normal and fine; drilling through five or six unrelated layers is the signal to reach for Context or a state management library instead.

6. Custom Hooks for Reusable Logic

A custom hook is simply a function whose name starts with 'use' that calls other hooks internally — it's the mechanism React provides for extracting stateful logic (not just plain functions, but logic that itself uses `useState/useEffect`) so it can be reused across multiple components without duplicating the underlying implementation. Unlike a regular utility function, each component that calls a custom hook gets its own independent state instance — the hook is not a shared singleton, it's a reusable recipe for a stateful behavior.

JSX

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);
    fetch(url).then((r) => r.json()).then((d) => {
      setData(d);
      setLoading(false);
    });
  }, [url]);

  return { data, loading };
}

// Reused across any component that needs remote data:
function StudentList() {
  const { data: students, loading } = useFetch("/api/students");
  if (loading) return <p>Loading...</p>;
  return <ul>{students.map((s) => <li key={s.id}>{s.name}</li>)}</ul>;
}
```

7. Component Design Patterns

Two patterns recur constantly in production React codebases. The container/presentational split separates data-fetching and logic (container) from purely visual rendering (presentational, ideally receiving everything via props and rendering the same output for the same input). Compound components — like a `<Tabs>` component whose `<Tabs.List>` and `<Tabs.Panel>` children implicitly coordinate through shared context — let you expose a flexible, composable API instead of one giant component with dozens of configuration props. Both patterns exist to keep individual components small, focused, and independently testable rather than growing into a single component that does everything.

Common Pitfalls

-
- Mutating state directly (`array.push(item)`; `setState(array)`) instead of creating a new reference, so React never detects the change and skips the re-render.
 - Using array index as the key prop for a list that can reorder, insert, or remove items — causing React to misattribute state and DOM nodes to the wrong items.
 - Omitting a value used inside `useEffect` from its dependency array, causing the effect to run with stale, captured values ('stale closure' bugs).
 - Reaching for `useEffect` to compute a value that's actually just a derived value from existing props/state, instead of calculating it directly during render.
 - Prop-drilling the same value through five or more intermediate components instead of introducing Context once it's clearly warranted.

- Forgetting the cleanup function in `useEffect` for subscriptions or timers, causing memory leaks or 'set state on unmounted component' warnings.
- Defining a new inline function or object literal as a prop on every render, which can defeat memoization (`React.memo`) on the child receiving it.

Key Takeaways

- React re-renders in response to new state or props references, not in-place mutation — always create new objects/arrays when updating state.
- `useEffect` is for synchronizing with something outside React (network, subscriptions, DOM APIs) — not for computing derived values.
- A stable, unique key (not array index, unless the list is static) is required for any list that can reorder or change length.
- Lift state to the nearest common ancestor when siblings need to share it; reach for `Context` only once prop drilling becomes genuinely painful.
- Custom hooks extract reusable stateful logic; each calling component still gets its own independent state.
- Small, composable components with a clear single responsibility scale far better than a few large, heavily-configured ones.

Learn More

React Docs — Describing the UI

<https://react.dev/learn/describing-the-ui>

React Docs — Synchronizing with Effects

<https://react.dev/learn/synchronizing-with-effects>

React Docs — Reusing Logic with Custom Hooks

<https://react.dev/learn/reusing-logic-with-custom-hooks>